Attention I: From Soft Lookup to Self-Attention

Lecture 5 · Large Language Models

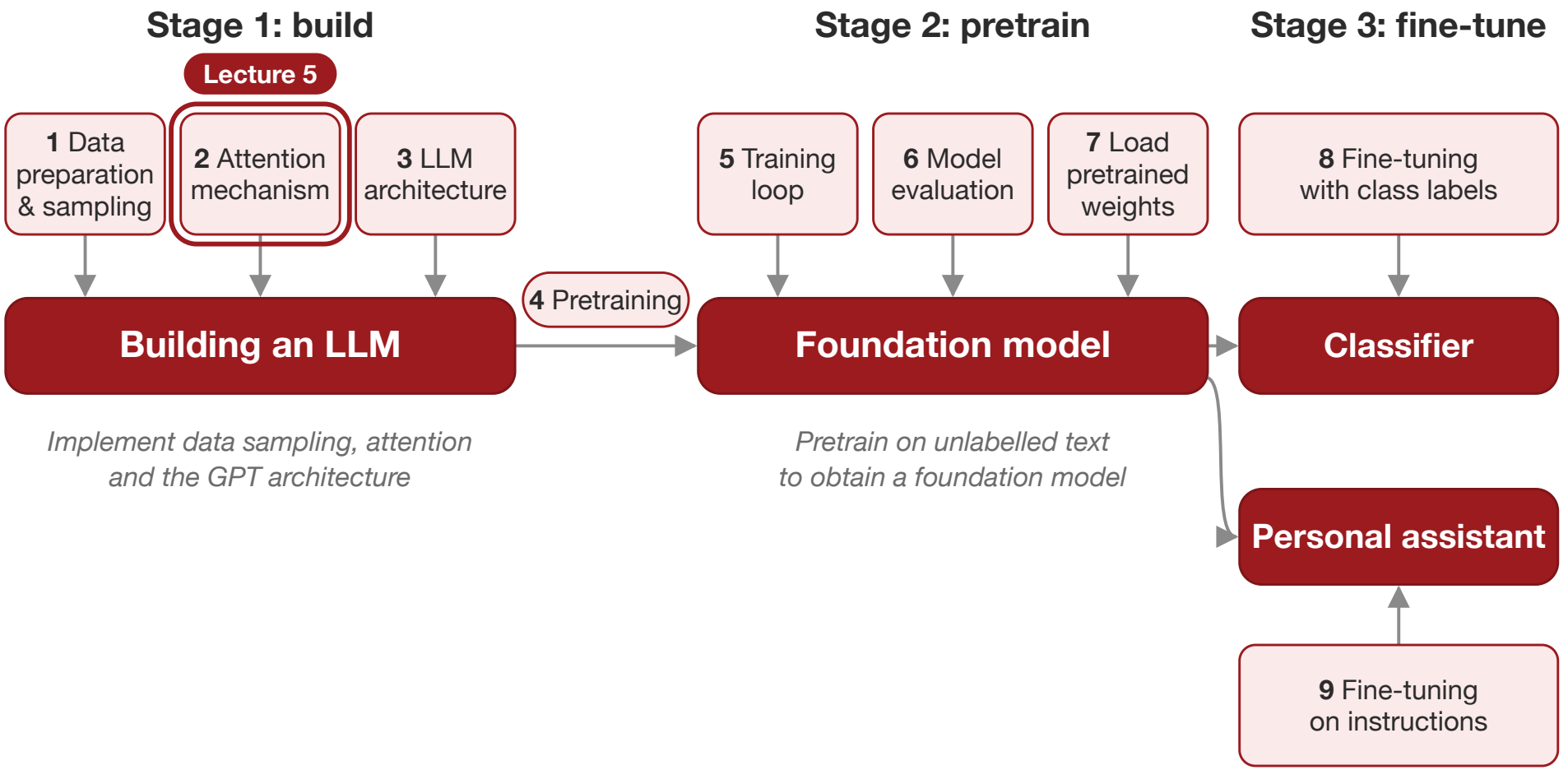
Daniel Kucharczyk
Wrocław University of Science and Technology

2026-11-06

Recap: every token now enters as a vector that knows its position

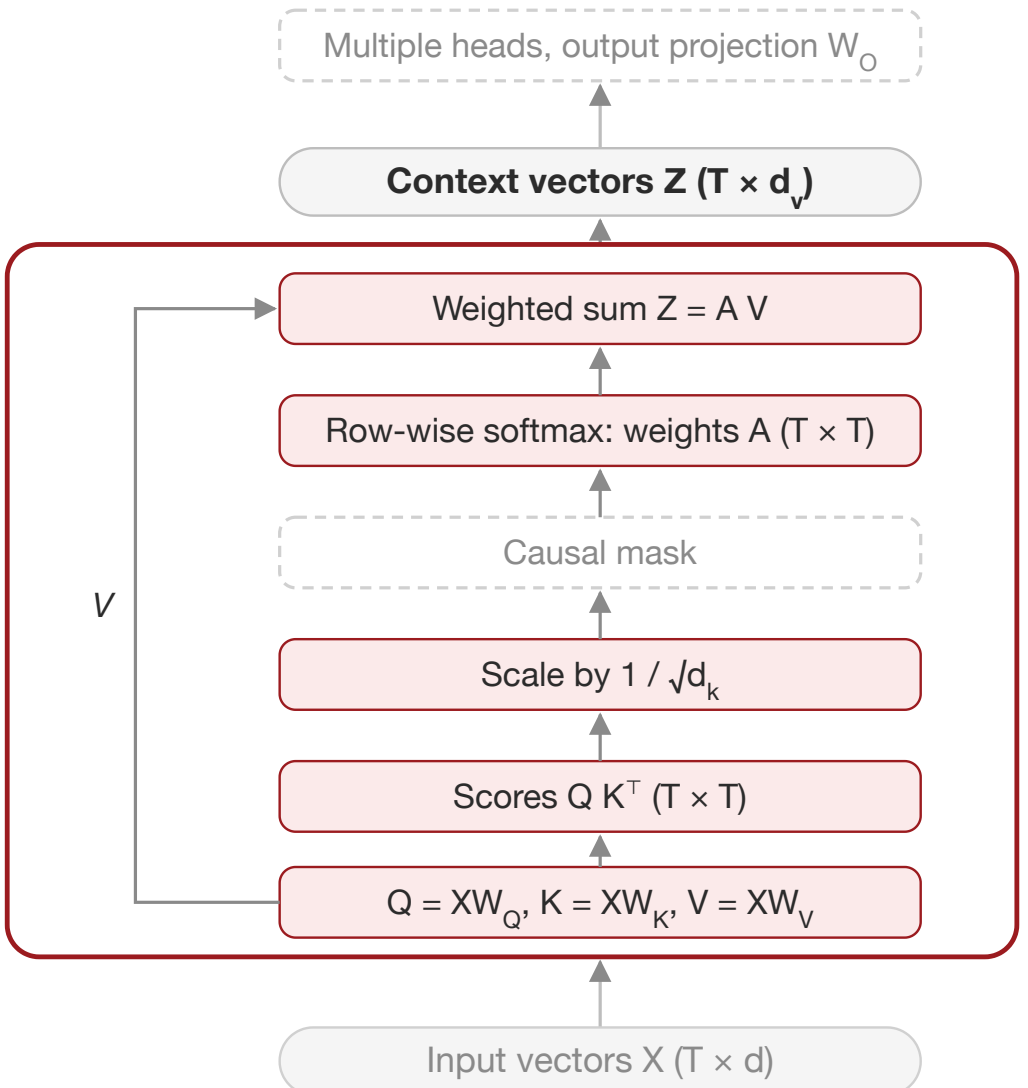
- Lecture 4: ID → token embedding $E_{x_t,:}$ + position vector.
- Box 1 delivers $X \in \mathbb{R}^{B \times T \times d}$.
- Each row was computed from **one** token alone.
- **Open question:** how does a token use its neighbours?

Lecture 5 opens box 2: the attention mechanism



Course roadmap with today's box outlined. Redrawn after Raschka (Raschka 2024).

Today: from input vectors to context vectors



Lecture 6

Today's focus

Parts 2–3: a weighted average of the values

Part 2: soft lookup

Lecture 6

Part 5: why √d_k

Part 3: scores, no weights

Part 4: queries, keys, values

Box 1, Lectures 3–4: tokens + positions

How does `bank` in `river bank` know it is about rivers?

- Lecture 4: one embedding per token, `bank` included.
- `river bank` and `bank account`: the same row of E.
- Fix: rebuild each vector from its context.
- Today: a weighted average with data-dependent weights.

Agenda

1. The long-sequence problem: the fixed-size bottleneck of encoder–decoder RNNs
2. Attention as a differentiable dictionary lookup and as a kernel smoother (Nadaraya–Watson)
3. Self-attention without trainable weights, then with query, key and value projections
4. Softmax as a smooth argmax; the attention matrix as a row-stochastic matrix
5. Why scale by $\sqrt{d_k}$: the variance of $q^\top k$ and softmax saturation

Learning objectives

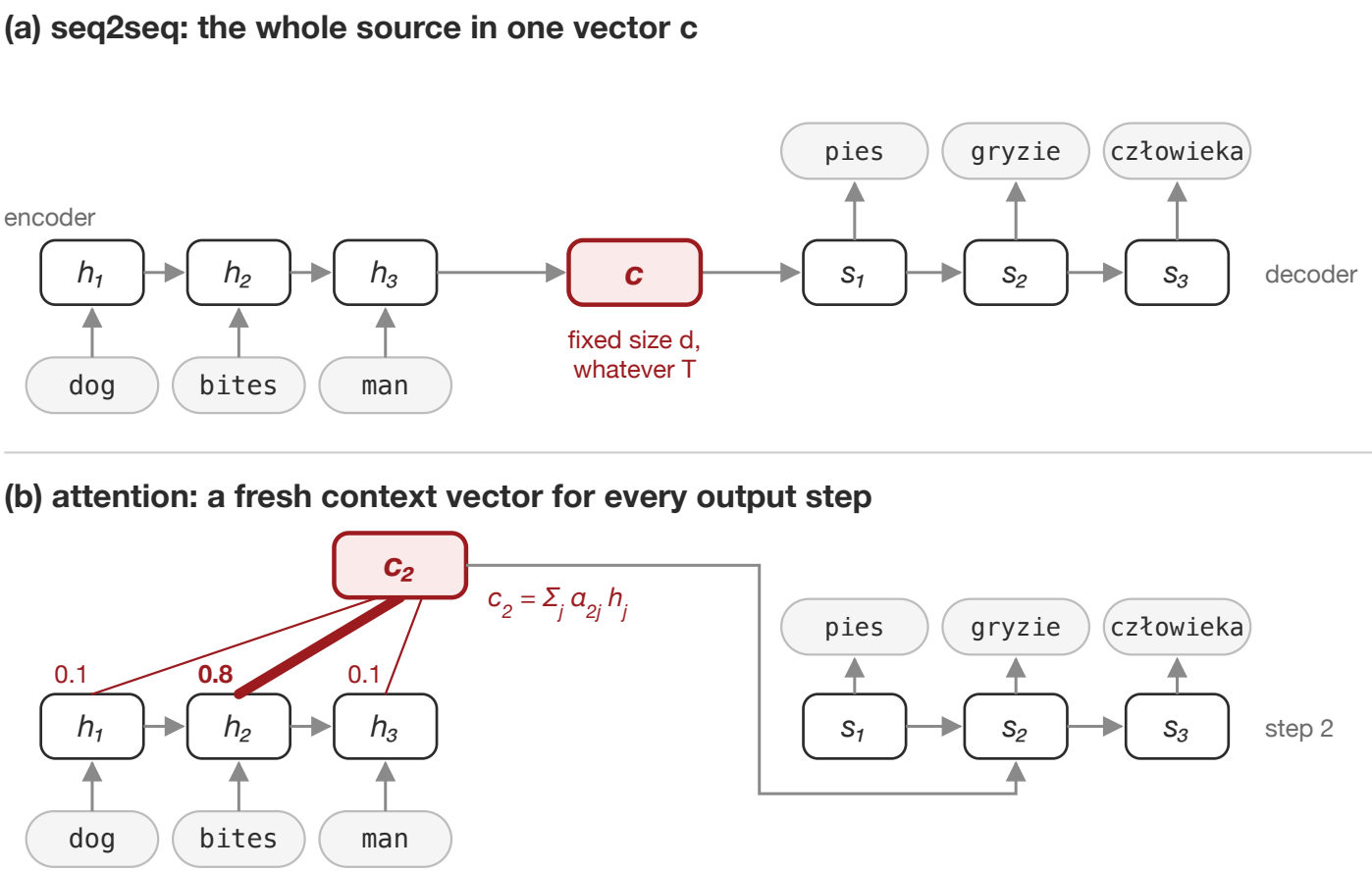
After this lecture you should be able to:

1. Compute attention weights and context vectors by hand for a three-token example.
2. Prove that $\mathrm{Var}(q^\top k) = d_k$ when the components are independent with zero mean and unit variance.
3. Show that self-attention without positional information is permutation-equivariant.
4. Interpret attention as kernel regression over the value vectors.

These objectives define the exam scope.

Part 1: One vector is a bottleneck

seq2seq squeezes the whole source sentence into one vector



Translating `dog bites man` into Polish. (a) One summary vector c. (b) A fresh c_i per output step. Redrawn after (Sutskever et al. 2014; Bahdanau et al. 2015).

Attention gives every output step its own weighted average

$$c_i = \sum_{j=1}^{T} \alpha_{ij}\, h_j, \qquad \alpha_{ij} = \mathrm{softmax}_j\left(\mathrm{score}(s_{i-1}, h_j)\right)$$

- h_j: encoder state of source token j; s_{i-1} : decoder state.
- Three steps: **score**, **normalise**, **average** (Bahdanau et al. 2015).
- No fixed-size summary: c_i is rebuilt for each i.
- Gradients reach every h_j directly, not through T steps.

Self-attention keeps the average and drops the RNN

- Bahdanau: decoder asks, encoder states answer.
- Self-attention: every token asks **all tokens of its own sequence**.
- Queries, keys and values all come from the same X.
- No recurrence: all T outputs in parallel (Vaswani et al. 2017).

Part 2: Attention is a soft dictionary lookup

A hard lookup returns exactly one value

$$\text{lookup}(q) = \sum_{s=1}^{T} \mathbf{1}[q = k_s]\ v_s$$

- Python: `d["bites"]` returns `v2`, nothing else.
- Weights are one-hot: $(0, 1, 0)$.
- A query that matches no key returns nothing.
- Not differentiable: the weight jumps from 0 to 1.

A soft lookup weights every value by its similarity to the query

$$\alpha_s = \frac{\exp\left(\text{score}(q, k_s)\right)}{\sum_{s'=1}^{T} \exp\left(\text{score}(q, k_{s'})\right)}, \qquad \text{out}(q) = \sum_{s=1}^{T} \alpha_s \, v_s$$

- Replace $\mathbf{1}[q = k_s]$ by a smooth score, e.g. $q^\top k_s$.
- Every value contributes; similar keys contribute more.
- Differentiable in q, every k_s and every v_s.

Softmax is a smooth argmax; the temperature sets how sharp

$$\mathrm{softmax}(a/\tau)_s = \frac{e^{a_s/\tau}}{\sum_{s'} e^{a_{s'}/\tau}}, \qquad a = (2, 1, 0)$$

τ	weights
0.25	$(0.982,\ 0.018,\ 0.000)$
1	$(0.665,\ 0.245,\ 0.090)$
4	$(0.419,\ 0.326,\ 0.254)$

- $\tau \to 0$: one-hot at the argmax (hard lookup); $\tau \to \infty$: uniform.

Attention outputs are convex combinations of the values

- $\alpha_s \geq 0$ and $\sum_s \alpha_s = 1$.
- So $\mathrm{out}(q)$ lies in the convex hull of $v_1, \ldots, v_T$.
- Each coordinate stays between the smallest and largest value.
- Weights depend on the input; an MLP's weights do not.

Nadaraya–Watson regression averages labels weighted by a kernel

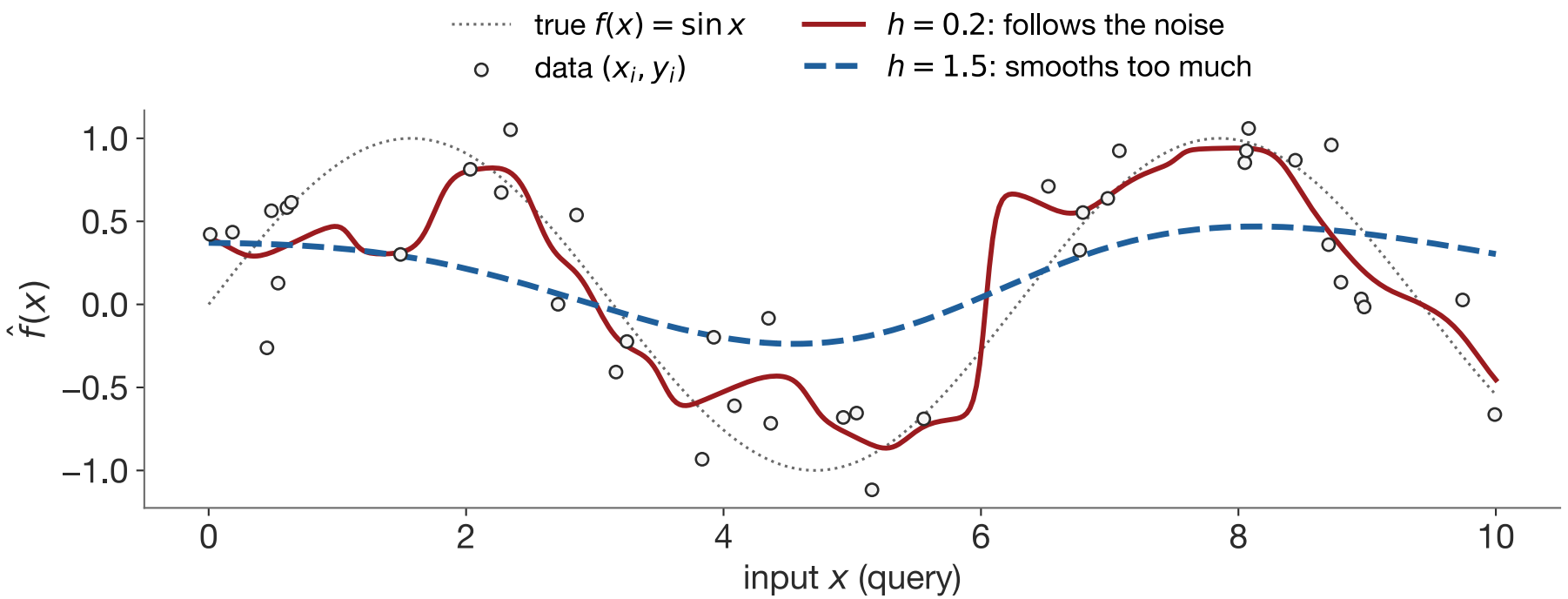
Definition (Nadaraya–Watson estimator)

Given data $(x_i, y_i)_{i=1}^n$ and a kernel $K(x, x') \geq 0$,

$$\hat{f}(x) = \sum_{i=1}^{n} \frac{K(x, x_i)}{\sum_{j=1}^{n} K(x, x_j)} \, y_i .$$

- Gaussian kernel: $K(x, x_i) = \exp\left(-(x - x_i)^2 / 2h^2 \right)$, bandwidth h.
- Prediction at x: average of the y_i with nearby x_i (Nadaraya 1964; Watson 1964).

The bandwidth controls how local the average is



Nadaraya–Watson regression with a Gaussian kernel on 40 noisy samples of $\sin x$.

- Small h: nearly a lookup of the nearest point. Large h: nearly the global mean.

Attention is Nadaraya–Watson regression with an exponential kernel

Proposition (objective 4)

With $K(q,k) = \exp\left(q^\top k/\sqrt{d_k}\right)$, query q_t in place of x, keys k_s in place of x_i and values v_s in place of y_i:

$$z_t = \sum_{s=1}^{T} \frac{K(q_t, k_s)}{\sum_{s'} K(q_t, k_{s'})} v_s = \sum_{s=1}^{T} \alpha_{ts}\, v_s\,.$$

- Proof: the ratio is $\mathrm{softmax}_s(q_t^\top k_s/\sqrt{d_k}) = \alpha_{ts}$.
- Values play the role of labels (Tsai et al. 2019).

With equal-length keys the exponential kernel is a Gaussian kernel

$$e^{-\|q-k\|^2/2} = e^{q^\top k}\ e^{-\|q\|^2/2}\ e^{-\|k\|^2/2}$$

- $e^{-\|q\|^2/2}$ is the same for every key: it cancels in the ratio.
- Equal $\|k_s\|$: softmax weights = Gaussian Nadaraya–Watson weights.
- Otherwise long keys get the extra factor $e^{\|k\|^2/2}$.

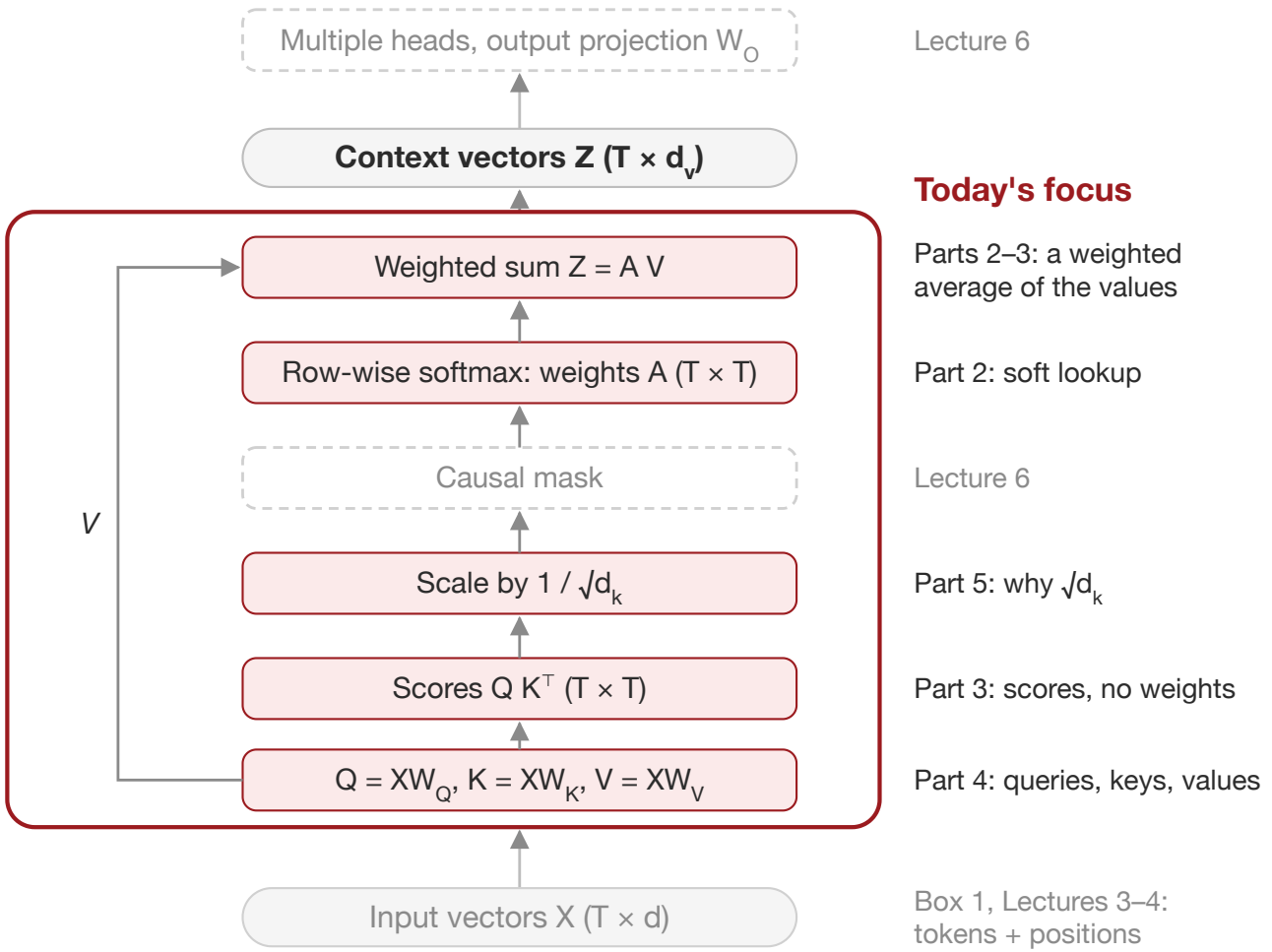
The learned attention kernel is not symmetric

> ⚠ **Pitfall: not a classical kernel**
>
> Queries and keys come from different matrices (Part 4), so the kernel between two **tokens** is $K(x_t, x_s) = \exp\left(x_t^\top W_Q W_K^\top x_s / \sqrt{d_k}\right) \neq K(x_s, x_t)$ in general.

- How much `dog` attends to `bites` ≠ how much `bites` attends to `dog`.
- Classical kernels are symmetric and positive semi-definite.
- The asymmetry is useful; Part 4 shows it in numbers.

Part 3: Self-attention without trainable weights

Part 3 skips the projections: scores straight from X



Today's steps; Part 3 uses $Q = K = V = X$ and no scaling.

Without weights, every token is its own query, key and value

$$S = XX^\top \in \mathbb{R}^{T\times T}, \qquad A = \mathrm{softmax}_{\mathrm{row}}(S), \qquad Z = AX \in \mathbb{R}^{T\times d}$$

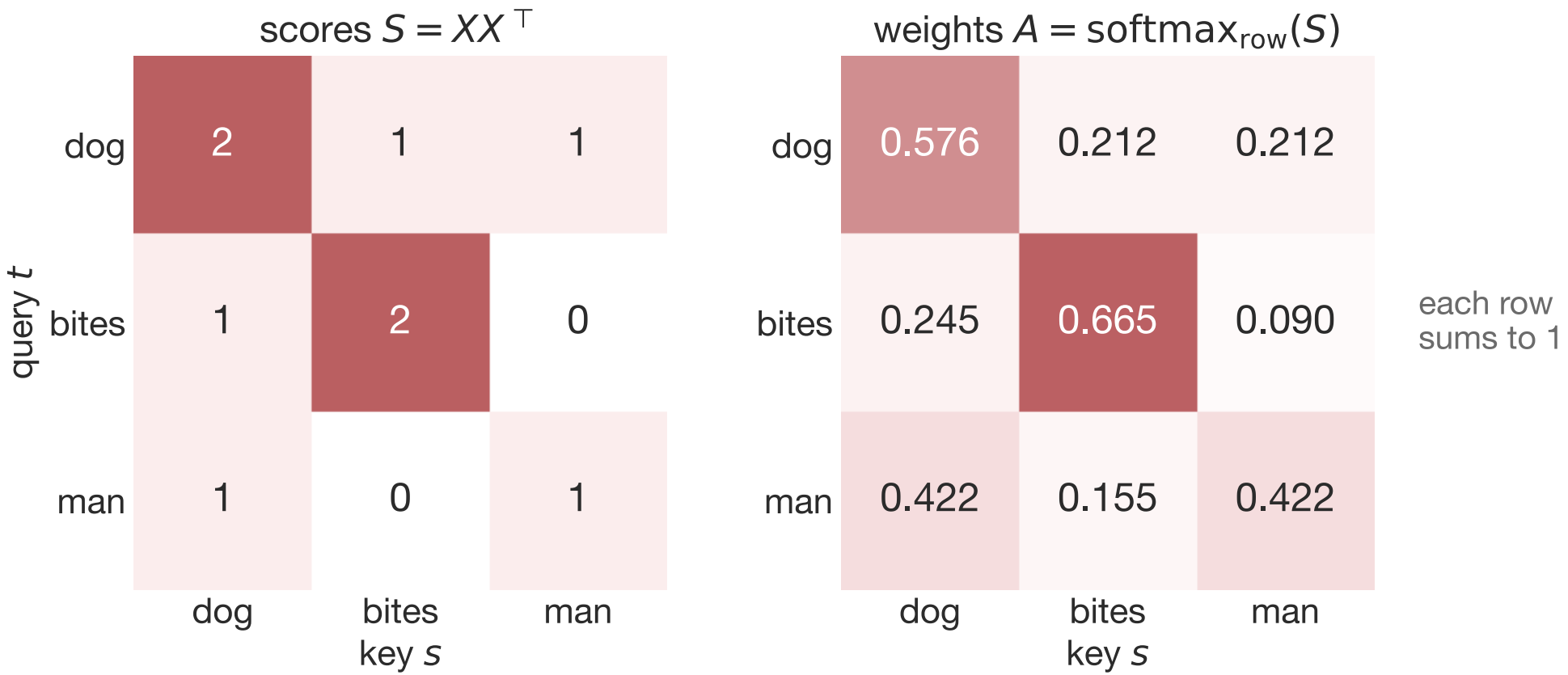
- $X \in \mathbb{R}^{T\times d}$: row t is $x_t^\top$.
- Score $a_{ts} = S_{ts} = x_t^\top x_s$: query t, key s.
- Weight $\alpha_{ts} = A_{ts}$; output $z_t = \sum_s \alpha_{ts}\, x_s$.

By hand: dog attends mostly to itself

$x_{\text{dog}} = (1, 1, 0),\ x_{\text{bites}} = (0, 1, 1),\ x_{\text{man}} = (1, 0, 0),\ d = 3$

- Scores: $\left(x_{\text{dog}}^{\top} x_s\right)_s = (2,\ 1,\ 1).$
- Weights: $\dfrac{(e^2,\ e,\ e)}{e^2 + 2e} \approx (0.576,\ 0.212,\ 0.212).$
- Context:
  $z_{\text{dog}} = 0.576\, x_{\text{dog}} + 0.212\, x_{\text{bites}} + 0.212\, x_{\text{man}} \approx (0.788,\ 0.788,\ 0.2$

All rows at once: A is a row-stochastic matrix



Scores and weights for `dog bites man`.

- $A \geq 0,\ A\mathbf{1} = \mathbf{1};\ Z = AX$: each row a convex combination.

Three lines of PyTorch reproduce the hand computation

```python
import torch

X = torch.tensor([[1., 1., 0.],   # dog
                  [0., 1., 1.],   # bites
                  [1., 0., 0.]])  # man        (T, d) = (3, 3)
S = X @ X.T                       # (T, T) scores
A = torch.softmax(S, dim=-1)      # (T, T) each row sums to 1
Z = A @ X                         # (T, d) context vectors
print(A.round(decimals=3))
print(Z[0].round(decimals=3), A.sum(dim=-1))
```

```
tensor([[0.5760, 0.2120, 0.2120],
        [0.2450, 0.6650, 0.0900],
        [0.4220, 0.1550, 0.4220]])
tensor([0.7880, 0.7880, 0.2120]) tensor([1.0000, 1.0000, 1.0000])
```

Reordering the tokens multiplies X by a permutation matrix

- Permutation matrix P: one 1 in every row and column.
- $(PX)_{t,:} = X_{\pi(t),:}$: row t of PX is row $\pi(t)$ of X.
- $P^\top P = I$.
- `man bites dog` $= PX$ with P swapping rows 1 and 3.

Row-wise softmax commutes with reordering rows and columns

Lemma

For every $S \in \mathbb{R}^{T \times T}$ and permutation matrix P: $\operatorname{softmax}_{\text{row}}(PSP^\top) = P \operatorname{softmax}_{\text{row}}(S) P^\top$.

- $(PSP^\top)_{ts} = S_{\pi(t)\pi(s)}$.
- Row t of $PSP^\top$: row $\pi(t)$ of S, entries reordered.
- Its normaliser $\sum_{s'} e^{S_{\pi(t)\pi(s')}} = \sum_{u} e^{S_{\pi(t)u}}$: same sum.
- Hence entry (t, s) is $\operatorname{softmax}(S)_{\pi(t)\pi(s)}$.

Without positions, self-attention is permutation-equivariant

Proposition (objective 3)

For every $X \in \mathbb{R}^{T \times d}$ and permutation matrix P: $\mathrm{SA}(PX) = P\,\mathrm{SA}(X)$.

$$\mathrm{SA}(PX) = \mathrm{softmax}_{\mathrm{row}}\left(PXX^\top P^\top\right) PX$$

$$\overset{\text{lemma}}{=} P\ \mathrm{softmax}_{\mathrm{row}}(XX^\top)\, P^\top PX = P\,\mathrm{SA}(X)$$

- Last step: $P^\top P = I$.

`man bites dog` gets the same context vectors, reordered

```python
P = torch.eye(3)[[2, 1, 0]]           # (T, T): swap dog and man

def sa(X):                            # (T, d) -> (T, d), no weights
    return torch.softmax(X @ X.T, dim=-1) @ X

print(torch.allclose(sa(P @ X), P @ sa(X)))
print(sa(P @ X).round(decimals=3))    # rows: man, bites, dog
```

```
True
tensor([[0.8450, 0.5780, 0.1550],
        [0.3350, 0.9100, 0.6650],
        [0.7880, 0.7880, 0.2120]])
```

Without positions, word order is invisible to attention

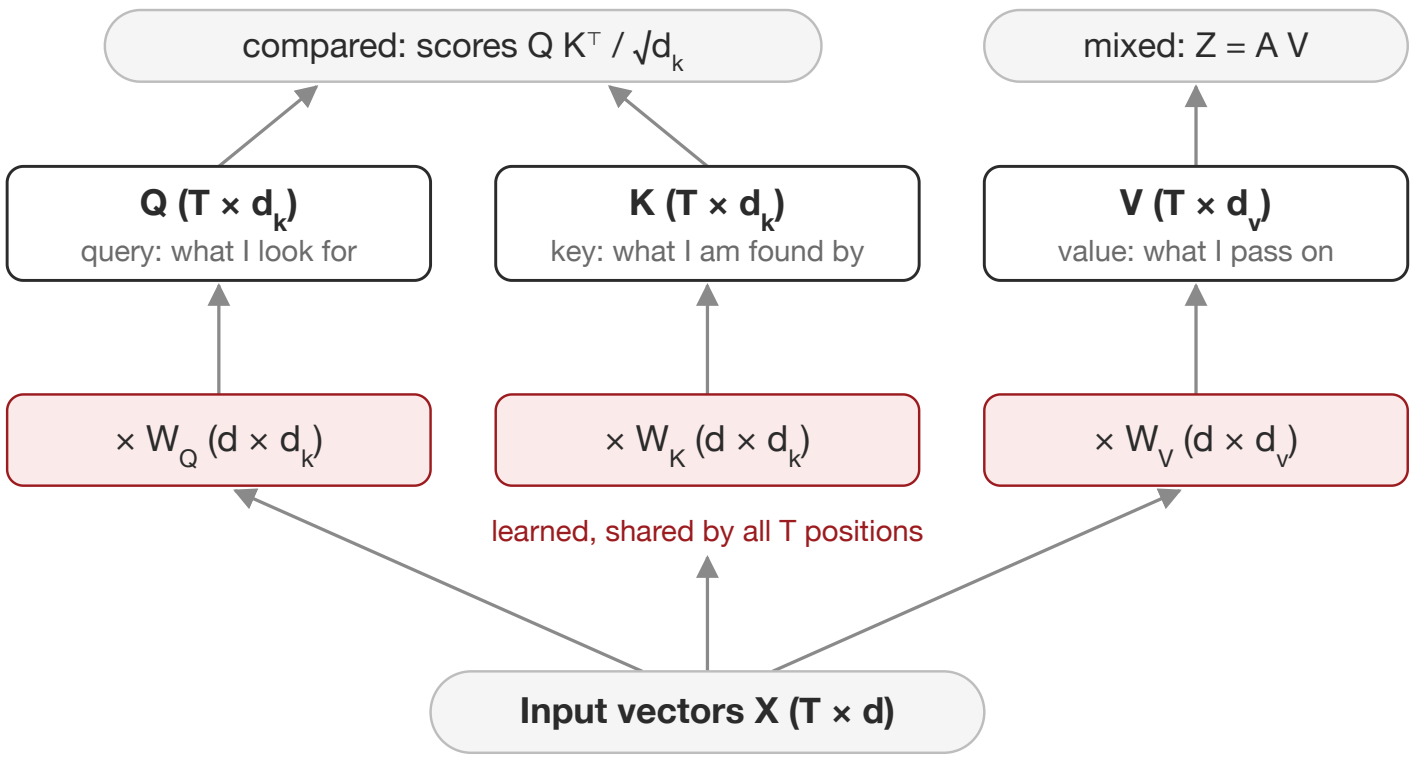
- Subject or object: dog gets the same z_{dog}.
- Lecture 4 adds a position vector to each row **before** attention.
- Reordering tokens then does not reorder the positions.
- So the reordered input is no longer PX.

Weight-free attention has three built-in limitations

- **Self-match:** $a_{tt} = \|x_t\|^2$ is often the largest score.
- **Symmetric scores:** $a_{ts} = a_{st}$.
- **One vector, three roles:** looking, being found, passing on.
- **Nothing to learn:** the weights follow from the embeddings alone.

Part 4: Queries, keys and values

Three learned projections give each token three roles



Queries, keys and values are three linear maps of the same input.

- $Q = XW_Q,\ K = XW_K,\ V = XW_V$: one row per token.

Notation: rows today, columns in Lecture 4, transposed in PyTorch

	query of token t	matrix shape
today (rows)	$q_t^\top = x_t^\top W_Q$	$W_Q : (d, d_k)$
Lecture 4 (columns)	$q_t = W_Q \; h_t$	(d_k, d)
`nn.Linear(d, d_k)`	`x @ weight.T`	`weight`: (d_k, d)

- Same map: Lecture 4's W_Q and `weight` equal today's $W_Q^\top$.
- Lab 5 checkpoint: copy `weight.T` into `nn.Parameter` to match.

Self-attention compares queries with keys and averages values

Definition (scaled dot-product self-attention)

For $X \in \mathbb{R}^{T \times d}$, $W_Q, W_K \in \mathbb{R}^{d \times d_k}$, $W_V \in \mathbb{R}^{d \times d_v}$ and $Q = XW_Q, K = XW_K, V = XW_V$:

$$\mathrm{Attn}(X) = \mathrm{softmax}_{\mathrm{row}}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V \in \mathbb{R}^{T \times d_v}.$$

- Shapes: $(T, d_k)(d_k, T) = (T, T)$, then $(T, T)(T, d_v) = (T, d_v)$.
- Row t: $\alpha_{ts} = \mathrm{softmax}_s\left(q_t^\top k_s / \sqrt{d_k}\right)$, $z_t = \sum_s \alpha_{ts} v_s$.

Vaswani et al. (2017), Section 3.2.1; Phuong and Hutter (2022).

Worked example: projections give each token a query, a key and a value

$$W_Q = \begin{pmatrix} 1 & 0 \\ 0 & 0 \\ 0 & 1 \end{pmatrix}, \quad W_K = \begin{pmatrix} 0 & 1 \\ 1 & 0 \\ 1 & 0 \end{pmatrix}, \quad W_V = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 0 & 1 \end{pmatrix}$$

token	x	$q = xW_Q$	$k = xW_K$	$v = xW_V$
`dog`	$(1, 1, 0)$	$(1, 0)$	$(1, 1)$	$(1, 1)$
`bites`	$(0, 1, 1)$	$(0, 1)$	$(2, 0)$	$(0, 2)$
`man`	$(1, 0, 0)$	$(1, 0)$	$(0, 1)$	$(1, 0)$

By hand: with projections, dog attends mostly to bites

$$QK^\top = \begin{pmatrix} 1 & 2 & 0 \\ 1 & 0 & 1 \\ 1 & 2 & 0 \end{pmatrix}$$ rows: queries dog, bites, man; columns: key

- dog row: $(1, 2, 0)/\sqrt{2} \approx (0.71,\ 1.41,\ 0)$ → weights $\approx (0.284,\ 0.576,\ 0.140)$.
- $z_{\text{dog}} \approx 0.284\,(1, 1) + 0.576\,(0, 2) + 0.140\,(1, 0) \approx (0.424,\ 1.436)$.
- Not symmetric: $a_{\text{dog,bites}} = 2$ but $a_{\text{bites,dog}} = 1$.

A functional `self_attention` in four lines

```python
def self_attention(X, W_q, W_k, W_v):          # X: (T, d) or (B, T, d)
    Q, K, V = X @ W_q, X @ W_k, X @ W_v        # (T, d_k), (T, d_k), (T, d_v)
    S = Q @ K.transpose(-2, -1) / K.shape[-1] ** 0.5  # (T, T)
    return torch.softmax(S, dim=-1) @ V        # (T, d_v) context vectors

W_q = torch.tensor([[1., 0.], [0., 0.], [0., 1.]])  # (d, d_k)
W_k = torch.tensor([[0., 1.], [1., 0.], [1., 0.]])  # (d, d_k)
W_v = torch.tensor([[1., 0.], [0., 1.], [0., 1.]])  # (d, d_v)
print(self_attention(X, W_q, W_k, W_v).round(decimals=3))
```

```
tensor([[0.4240, 1.4360],
        [0.8020, 0.7970],
        [0.4240, 1.4360]])
```

The projections hold $d(2d_k + d_v)$ parameters, whatever T

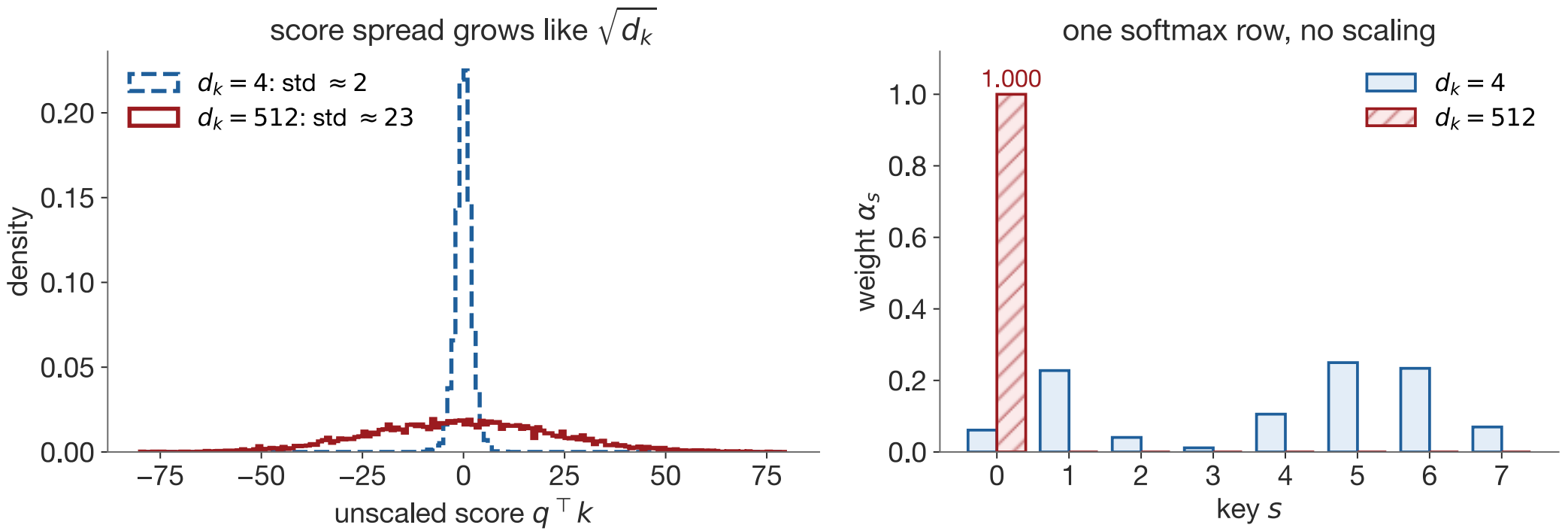
- W_Q and W_K: $d \cdot d_k$ each; W_V: $d \cdot d_v$.
- Toy example: $3 \cdot (2 \cdot 2 + 2) = 18$.
- $d = d_k = d_v = 768$: $3 \cdot 768^2 = 1{,}769{,}472$.
- Same matrices for every position and every length T.

Projections act row by row, so permutation equivariance survives

- $(PX)W = P(XW)$: Q, K, V are permuted like X.
- $\mathrm{Attn}(PX) = \mathrm{softmax}\left(PQK^\top P^\top/\sqrt{d_k}\right)PV = P\,\mathrm{Attn}(X)$.
- Same lemma, same proof as in Part 3.
- Learned weights do not create order: positions are still needed.

Part 5: Why scale by $\sqrt{d_k}$

Without scaling, large d_k saturates the softmax



Random q, k with independent standard normal entries. Left: spread of $q^\top k$. Right: one softmax row over 8 keys.

- $d_k = 512$: score gaps of tens, so e^{gap} makes one key win.

The dot product of random vectors has variance d_k

Proposition (objective 2)

Let $q_1, \ldots, q_{d_k}, k_1, \ldots, k_{d_k}$ be independent with mean 0 and variance 1. Then $\mathbb{E}[q^\top k] = 0$ and $\mathrm{Var}(q^\top k) = d_k$.

- $\mathbb{E}[q_i k_i] = \mathbb{E}[q_i]\,\mathbb{E}[k_i] = 0.$
- $\mathrm{Var}(q_i k_i) = \mathbb{E}[q_i^2 k_i^2] - 0^2 = \mathbb{E}[q_i^2]\,\mathbb{E}[k_i^2] = 1 \cdot 1.$
- The d_k terms $q_i k_i$ are independent: variances add up to d_k.

Dividing by $\sqrt{d_k}$ brings the variance back to 1

- $\mathrm{Var}(cY) = c^2\,\mathrm{Var}(Y)$ with $c = 1/\sqrt{d_k}$: $\mathrm{Var}\left(q^\top k/\sqrt{d_k}\right) = 1.$

```python
torch.manual_seed(0)
for d_k in [4, 64, 512]:
    q = torch.randn(10_000, d_k)                    # (N, d_k)
    k = torch.randn(10_000, d_k)                    # (N, d_k)
    s = (q * k).sum(dim=-1)                         # (N,) dot products
    s_scaled = s / d_k ** 0.5                       # (N,) scaled scores
    print(d_k, round(s.var().item(), 1), round(s_scaled.var().item(), 3))
```

```
4 4.0 0.992
64 64.1 1.002
512 505.8 0.988
```

A saturated softmax passes almost no gradient

$$\frac{\partial p}{\partial a} = \mathrm{diag}(p) - pp^{\top}, \qquad p = \mathrm{softmax}(a)$$

- Entry (s, s'): $p_s\,(\delta_{ss'} - p_{s'})$.
- $p \to$ one-hot: every entry $\to 0$.
- $p = (0.98, 0.02, 0)$: largest entry ≈ 0.02; uniform: $2/9 \approx 0.22$.
- Saturated scores mean W_Q, W_K barely learn.

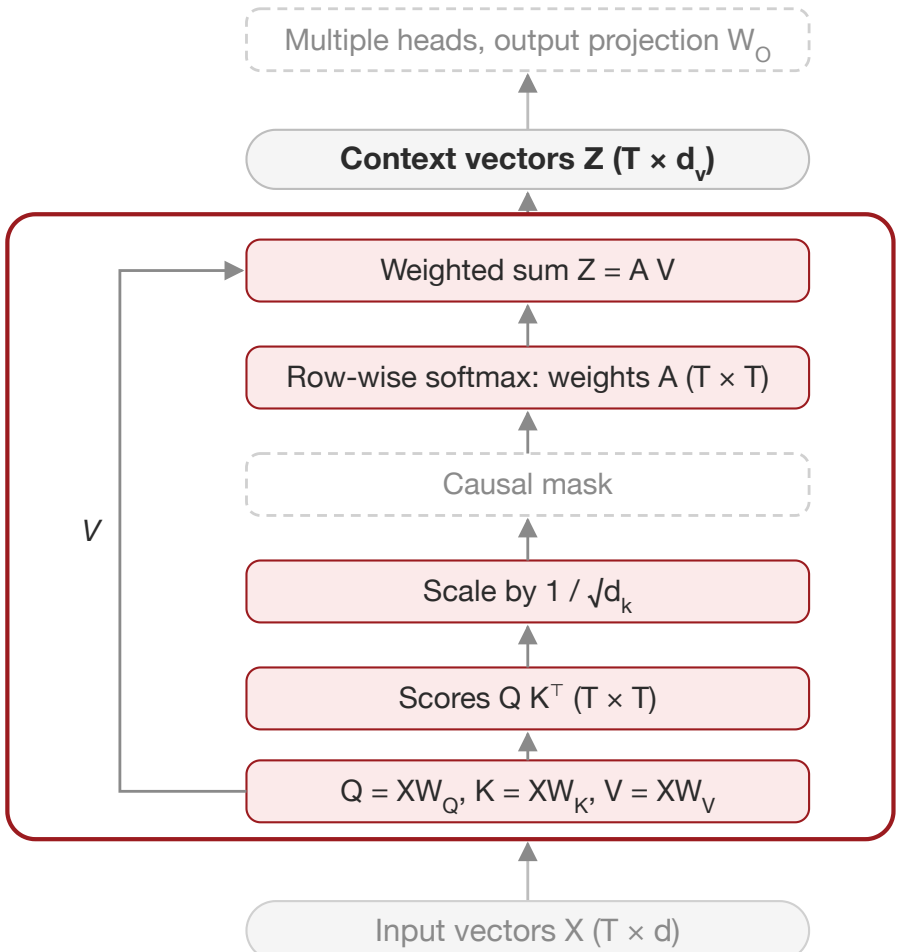
$\sqrt{d_k}$ fixes the scale at initialisation, not forever

> ⚠ **Pitfall: the assumptions hold only at the start**
>
> After training, q and k are neither independent nor unit-variance; scores can grow, and attention can become sharp again.

- $1/\sqrt{d_k}$ is a fixed temperature $\tau = \sqrt{d_k}$ (Part 2).
- Sharp attention can still be learned, by growing W_Q, W_K.
- Lab 5 Task 5.3: attention entropy against d_k.

Part 6: Wrap-up

Box 2 so far: project, score, scale, normalise, average



Lecture 6

Today's focus

Parts 2–3: a weighted average of the values

Part 2: soft lookup

Lecture 6

Part 5: why √d_k

Part 3: scores, no weights

Part 4: queries, keys, values

Box 1, Lectures 3–4: tokens + positions

Today's steps, all covered; the dashed steps are Lecture 6.

Context vectors leave today's box; two gaps remain for Lecture 6

- Output: $Z \in \mathbb{R}^{B \times T \times d_v}$, one context vector per token.
- **Gap 1:** token t also sees tokens $s > t$, the future → causal mask.
- **Gap 2:** one set of weights, one notion of relevance → multiple heads.
- `self_attention` already runs on (B, T, d) batches.

Lab 5 turns Parts 3–5 into code

- 5.1 (Part 3) Simplified attention: loops, then $\mathrm{softmax}(XX^\top)X$.
- 5.2 (Part 4) `SelfAttention_v1` (`nn.Parameter`), `SelfAttention_v2` (`nn.Linear`).
- 5.3 (Part 5) Attention entropy against d_k, with and without scaling.
- Checkpoint: v1 and v2 give equal outputs (notation slide).

Lab 5: Self-Attention Step by Step · Mon 9 Nov 2026 · List 3 due Sun 29 Nov 2026, 23:59

Key takeaways

- **Obj. 1:** score → softmax → weighted sum; dog: $\mathrm{softmax}(2, 1, 1) \approx (0.576, 0.212, 0.212)$.
- **Obj. 2:** $\mathrm{Var}(q^\top k) = d_k$, so divide by $\sqrt{d_k}$ to avoid saturation.
- **Obj. 3:** $\mathrm{SA}(PX) = P\ \mathrm{SA}(X)$: attention alone is order-blind.
- **Obj. 4:** attention = Nadaraya–Watson with $K(q, k) = e^{q^\top k/\sqrt{d_k}}$, values as labels.
- A is row-stochastic; $W_Q \neq W_K$ makes relevance learnable and asymmetric.

Mandatory resources

- **BLLM Ch 3.1–3.4:** Coding Attention Mechanisms (Raschka 2024).
- Vaswani et al. (2017): Attention Is All You Need, Section 3.2.1 (Vaswani et al. 2017).
- Phuong & Hutter (2022): Formal Algorithms for Transformers (Phuong and Hutter 2022).

Supporting resources

- Bahdanau et al. (2014): Neural Machine Translation by Jointly Learning to Align and Translate (Bahdanau et al. 2015).
- Tsai et al. (2019): Transformer Dissection (Tsai et al. 2019).
- Weng (2018): “Attention? Attention!” (Weng 2018).

References

Bahdanau, Dzmitry, Kyunghyun Cho, and Yoshua Bengio. 2015. “Neural Machine Translation by Jointly Learning to Align and Translate.” *International Conference on Learning Representations*. https://arxiv.org/abs/1409.0473.

Nadaraya, E. A. 1964. “On Estimating Regression.” *Theory of Probability and Its Applications* 9 (1): 141–42. https://doi.org/10.1137/1109020.

Phuong, Mary, and Marcus Hutter. 2022. *Formal Algorithms for Transformers*. https://arxiv.org/abs/2207.09238.

Raschka, Sebastian. 2024. *Build a Large Language Model (from Scratch)*. Manning. https://www.manning.com/books/build-a-large-language-model-from-scratch.

Sutskever, Ilya, Oriol Vinyals, and Quoc V. Le. 2014. “Sequence to Sequence Learning with Neural Networks.” *Advances in Neural Information Processing Systems* 27. https://arxiv.org/abs/1409.3215.

Tsai, Yao-Hung Hubert, Shaojie Bai, Makoto Yamada, Louis-Philippe Morency, and Ruslan Salakhutdinov. 2019. “Transformer Dissection: An Unified Understanding for Transformer’s Attention via the Lens of Kernel.” *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, 4344–53. https://arxiv.org/abs/1908.11775.

Vaswani, Ashish, Noam Shazeer, Niki Parmar, et al. 2017. “Attention Is All You Need.” *Advances in Neural Information Processing Systems* 30. https://arxiv.org/abs/1706.03762.

Watson, Geoffrey S. 1964. “Smooth Regression Analysis.” *Sankhyā: The Indian Journal of Statistics, Series A* 26 (4): 359–72.

Weng, Lilian. 2018. *Attention? Attention!* Lil’Log (blog post). https://lilianweng.github.io/posts/2018-06-24-attention/.

Questions?

Daniel Kucharczyk

- Wrocław University of Science and Technology
- Course repository
- Materials and submissions: ePortal
- Next: **Attention II: Causality & Multiple Heads** (Fri 13 Nov 2026).